

Feature: Client access: provisioning, and closing signup

From build-plan: feature 3b

Branch: feature/03b-client-access-provisioning-and-closing-signup

Status: verified. Whole feature seen and agreed by Frank 2026-09-30; steps 3b.1 to 3b.3 built, checked and reviewed step by step; step 3b.3's review findings F-48 to F-51 fixed 2026-09-30, F-49 by the claim table on Frank's choice. The checkpoint for the final review.

Branch named in the workspace's `feature/NN-name` form, as feature 3's was, so one number finds the branch, the archive (`03b-...md`) and the tag (`item-03b-done`).

Goal

The agency can turn a signed contract into a working login. The platform admin fills in one form (the business's name, the client's name, the client's email) and the API creates the client's login and the client's business together, with the client as its business owner and its first person. The client then signs in at `/sign-in` with a code sent to that email and lands straight in their own business. The platform admin is never a member of it.

The second half closes the two doors that are still open beside it: nobody, the platform admin included, creates a business through Better Auth's own `/organization/create` any more, and owners cannot invite people, because there is no invitation flow. Self-signup itself was already closed on 2026-09-23 (`disableSignUp: true`), so this feature does not touch the sign-in form's behaviour.

Today an onboarded client cannot reach the product at all. Nothing except this feature lets them, which makes it a prerequisite for Phase 5 and any client-facing deploy.

In scope

- `POST /admin/clients` , a platform-admin-only route that creates the client's login and business in one request, through Better Auth's own pieces (`auth.api.createUser` from the `admin` plugin, `auth.api.createOrganization` from the `organization` plugin), with no request headers, so Better Auth treats both as server actions on behalf of the client.
- If the business cannot be created after the login was, the login this request made is removed again. No request ever leaves a login with no business behind.
- `requirePlatformAdminMiddleware` , the one gate for `/admin/*` routes: `user.role === "admin"` on the session, the exception the coding standards already name.
- One shared Zod schema for the form and the route.
- Closing Better Auth's HTTP business creation for everyone (`allowUserToCreateOrganization: false`), and owners' and org admins' invitation permissions (`invitation: []`), closing F-16's first

half. F-16's second half is a comment correction: the platform admin can set `user.role` through the admin plugin's `/admin/set-role`, which the comments deny.

- The admin screen `/admin/clients/new`: the form, its states, and the success card. Built on `shadcn` `Input` and `Label` with `react-hook-form`, per the Forms standard (the first form to follow it; see F-14).
- A link to that screen on the dashboard, shown to the platform admin only.
- The signed-in-with-no-business screen (F-13): an ordinary user is told their login has no business yet and to contact the agency, with Sign out; the platform admin is sent to `/admin/clients/new`.
- Deleting `/create-organization`, which can only refuse once HTTP creation is closed.
- The comment corrections F-12 lists in the files this feature touches.

Out of scope

- **A welcome or invitation email.** There is no email transport until feature 6 (Resend). The success card tells the platform admin to let the client know. Production sign-in itself cannot send a code until feature 6 (`send-login-code.ts` throws in production); unchanged here.
- **An owner adding their own staff logins**, invitations, a second member in one business. Crew sign-in is named, not planned.
- **Listing, editing, suspending or deleting clients**, moving a client's plan, and an audit trail of who set up whom. Feature 23, the admin area.
- **The client's services and hours.** The client sets them in the app (feature 12); feature 10 decides how the agency's own business gets its first ones.
- **The dev seed.** `admin@example.com` still owns Summit Painting (dev) in development. That is a fixture, not a client, and Frank's calendar tests run on it. Unchanged.
- **Self-serve signup and business creation.** Feature 25 reopens both, in one edit, together with `organizationLimit`.
- **React Query.** One mutation with no cached list; the standard's React Query settings apply when the first list arrives (feature 11).
- Better Auth's own `/admin/create-user`, `/admin/set-role` and the rest of the admin plugin's HTTP routes stay as they are: platform admin only, and not used by the app.

Build loop

`workflow.stepReview` is `every` and `workflow.checkpointCommits` is `enabled`.

Each step's plan (Part 1, what it builds; Part 2, Done when) is gone through with Frank and gets its own yes just before it is built. After that yes the step runs straight through without asking: build, tests and checks, tick the box here, write the build log entry and push buildlogs, commit (`feat: 3b.N <what>`) and push to the feature branch, `/audit` scoped to the step, then the independent review. The one planned

stop is after the review, where its findings are talked through. P0/P1 are fixed before the next step, or Frank accepts them with a reason; P2/P3 are recorded and carried.

Only three things stop a step earlier: the agreed plan turns out wrong while building, a line only Frank crosses (a package, Railway or real data, `main` , a merge, a force push, deleting anything), or blocking review findings.

`/complete` runs its own final review over the already-reviewed steps, then merges with a merge commit on Frank's yes.

Build steps

- ☒ **3b.1 The provisioning route.** One request makes the client's login and business, with the client as business owner and first person.
 - **Blocker, answered by Frank 2026-09-30: A, refuse.** An email that already has a login is `409 email_taken` ; one login is one business for now. Rejected, B: add the business to that login (a typo hands a business to the wrong person, and they pick a business at every sign-in).
 - **Added by Frank 2026-09-30: running the setup again finishes it.** An email whose login belongs to no business, and whose `role` is not `"admin"` , can only be a setup that did not finish (a crash between the two creates, where the clean-up never ran). It is not refused: the business is made under that existing login instead. An email whose login belongs to any business is still `409 email_taken` . The platform admin's login is never reused. The clean-up after a failed business stays, as the second safety net.
 - `packages/shared/zod-validation/admin-validation-schemas/provision-client-validation-schema.ts` : `businessName` (the existing business-name rule, reused, not copied), `clientName` (trimmed, 1 to 100 characters), `clientEmail` (the sign-in email rule: trimmed, lowercased, valid). Exported through the package's existing `zod-validation` entry. Tests beside it.
 - `backend/middleware/auth-middleware/require-platform-admin-middleware.ts` : reads the session with `auth.api.getSession` ; none is `401 unauthenticated` ; a user whose `role` is not `"admin"` is `403 forbidden` . Not organization-scoped: the platform admin may belong to no business.
 - `backend/lib/admin/provision-client.ts` , one exported function: 1. slug from `toSlug(businessName)` ; empty is `400 bad_request` 2. the email already has a login: `409 email_taken` when that login belongs to any business or is the platform admin; otherwise it is an unfinished setup, so step 4 is skipped and its user id is used 3. the slug is taken: `409 slug_taken` 4. `auth.api.createUser({ body: { email, name } })` , no headers 5. `auth.api.createOrganization({ body: { name, slug, userId } })` , no headers, so the client is the creator and `creatorRole: "owner"` makes them business owner; the existing `afterCreateOrganization` hook makes the first person and links it to them 6. if 5 throws, the login made in 4 (never one reused in 2) is deleted (cascade takes its sessions and accounts) and the answer is `409 slug_taken` when the slug turned out taken in between, otherwise the error is

rethrown (500). A failed delete is logged with the user id only. The call site carries the comment the build plan asks for: these two server calls skip Better Auth's own admin checks because they carry no headers, so this function must only ever run behind `requirePlatformAdminMiddleware`, and must never be handed the request's headers (that would make the platform admin the creator, and so the business owner).

- `backend/routes/admin-routes.ts`: `POST /clients`, mounted at `/admin` in `app.ts` with `dashboardCorsMiddleware`, `dashboardCsrfMiddleware` and `dashboardNoStoreMiddleware` on `/admin/*`, then `requirePlatformAdminMiddleware`. Validates the body with the shared schema before any database call (a failed parse is `400 bad_request`; malformed JSON is Hono's own plain 400, F-36). Answers `201`.
- `backend/lib/errors/refuse.ts`: adds `email_taken` and `slug_taken`.
- `backend/routes/admin-routes.test.ts` against the local `scheduleads_dev`, signing in the real way as the existing route tests do, cleaning up its own rows.
- **Done when** the backend and shared tests pass (`npm run test --workspace=@scheduleads-app/shared`, `npm run test --workspace=backend`) and the backend builds, with tests proving: no session is 401; `owner@example.com` is 403; a bad body is 400; the platform admin gets 201 and the database then holds a login for the client (`role user`), a business with that slug on plan `agency`, the client as its only member with role `owner`, its first person linked to the client, and no membership for the platform admin; an existing email is 409 `email_taken` and creates nothing; a login with no business (made directly in the test) gets its business on a second setup, keeping its user id, while the platform admin's email is still `email_taken`; a taken slug is 409 `slug_taken` and leaves no login behind; a business creation forced to fail (`vi.spyOn(auth.api, "createOrganization")`) leaves no login behind; and the new client signs in with a code and `GET /me` answers their business with role `owner`.
- ☒ **3b.2 Close the old doors.** Only the provisioning route makes a business, and nobody can invite.
 - `backend/lib/auth/auth-server.ts`: `allowUserToCreateOrganization: false` with its comment rewritten (no one creates a business over HTTP; the platform admin provisions through `POST /admin/clients`; feature 25 reopens it together with `disableSignUp` and `organizationLimit`); `invitation: []` for `owner` and `admin`; the `user.role` comment says the admin plugin's `/admin/set-role` can set it (F-16); the `afterCreateOrganization` comment no longer says "today the platform admin".
 - The F-12 comment lines in `auth-server.ts` and `sign-in/page.tsx` and the shared schema comments that claim server-side use, corrected to the code.
 - **Carried from step 3b.1's review (Frank, 2026-09-30):**
 - F-36: Data / contracts says malformed JSON is Hono's own plain 400, and a missing or wrong body is `400 bad_request`.
 - F-37: tests that `/admin` answers carry `Cache-Control: no-store` and that a form-encoded post from another origin with the platform admin's cookie is refused and makes nothing.
 - F-39: finishing an unfinished setup writes the client's name just typed over the one on the reused login, and the answer returns it.

- F-41: force a failing `createUser` and read the log; if the email shows, log a safe reason instead and rethrow without the query's parameters.
- **Done when** backend tests prove: the platform admin's own `POST /api/auth/organization/create` is 403 and creates nothing; an owner's `POST /api/auth/organization/invite-member` is 403; and every 3b.1 test still passes, so provisioning is unaffected; the F-37 tests pass; a retried setup's answer and stored login carry the new name (F-39); and a forced `createUser` failure leaves no email in the log (F-41). F-16, F-36, F-37, F-39 and F-41 set to `fixed`.
- ☒ **3b.3 The admin screen, and nobody stuck.** The platform admin sets up a client in the browser; a login with no business is told what to do.
 - `npm shadcn add input label` into `frontend/components/ui/`. If the CLI wants to install any package, stop and ask before it does.
 - `frontend/app/admin/clients/new/page.tsx` and `frontend/components/admin/new-client-form.tsx`: react-hook-form with the shared schema through `@hookform/resolvers/zod`, `shadcn Input` and `Label`, the existing `AuthCard` shell.
 - `frontend/lib/api-client.ts`: `provisionClient()` through `dashboardApiClient.admin.clients.$post`, one result state per screen, as `fetchMe` does.
 - `frontend/app/page.tsx`: the dashboard shows "Set up a client" to the platform admin only; `PickOrganization` with an empty list shows the no-business card (ordinary user) or goes to `/admin/clients/new` (platform admin), never `/create-organization`.
 - `frontend/app/create-organization/page.tsx` deleted, and the now unused pieces of `auth-card.tsx` with it if nothing else uses them.
 - F-40, carried from step 3b.1's review: the business-name rule and the email rule each move to a file of their own name, and the business-name message reads for either a client's business or your own.
 - **Done when** `npm run build --workspace=frontend` and `npm run lint --workspace=frontend` pass, and in the browser against the local API: the platform admin sets up a test client and sees the success card; that client signs in with the code from the API's console and lands on their own business as owner, with the calendar card showing their own person; `owner@example.com` opening `/admin/clients/new` sees the not-for-you card; an email already in use shows its message under the email field. The test client is removed afterwards. F-13 and F-40 set to `fixed`.

Files / areas

- `packages/shared/zod-validation/admin-validation-schemas/` (new), `packages/shared/zod-validation/index.ts`, the business-name schema it reuses, and the sign-in email schema
- `backend/middleware/auth-middleware/require-platform-admin-middleware.ts` (new)
- `backend/lib/admin/provision-client.ts` (new)

- `backend/routes/admin-routes.ts` and `admin-routes.test.ts` (new)
- `backend/app.ts`, `backend/lib/errors/refuse.ts`, `backend/lib/auth/auth-server.ts`
- `frontend/app/admin/clients/new/page.tsx`, `frontend/components/admin/new-client-form.tsx`, `frontend/components/ui/input.tsx`, `frontend/components/ui/label.tsx` (new)
- `frontend/app/page.tsx`, `frontend/lib/api-client.ts`, `frontend/components/auth-card.tsx`, `frontend/app/sign-in/page.tsx` (comments only)
- `frontend/app/create-organization/page.tsx` (deleted)

Data / contracts

One migration, `0005_client_setup_claim` (step 3b.3's review, F-49): the `client_setup_claim` table below. Every other table already exists.

POST /admin/clients

- Dashboard route: dashboard CORS (only `APP_ORIGIN`, with the cookie), the origin check, `Cache-Control: no-store`.
- Who: a session whose user has `role === "admin"`, read from Better Auth's session on the server. No business is required or read; the route is not organization-scoped and takes no organization id from anyone.
- Body (JSON): `{ businessName: string, clientName: string, clientEmail: string }`, parsed with `provisionClientValidationSchema`. The slug is derived on the server with `toSlug(businessName)`, never sent.
- `201: { organization: { id, name, slug }, client: { id, name, email } }`.
- Refusals, all in the existing `{ error: { code, message } }` shape: `401 unauthenticated`, `403 forbidden` (not the platform admin), `400 bad_request` (a missing or wrong body, a failed schema, a name with no letters or digits; malformed JSON gets Hono's own plain-text 400, because the body is checked by Hono's validator, which types it for `AppType`), `409 email_taken`, `409 slug_taken`, `409 setup_in_progress`. Anything else is a 500 with no detail.
- Idempotency: a second identical request after a success is `409 email_taken`. After a setup that died between its two creates, the same request finishes it (the login with no business is reused). The form disables its button while sending.
- Never in an answer or a log: login codes, the client's email in a log line, request headers.

What a provisioned client is

- `user`: the email lowercased, the name given, `role "user"` (the admin plugin's default), `emailVerified` false until their first sign-in. No password, no `account` row: they sign in by email code only.
- `organization`: the name given, the derived slug, `plan agency` (the column default; nothing sends a plan).
- `member`: exactly one, the client, role `owner`.

- `resource` : the first person, named after the business, `kind person` , `userId` the client's. Made by the existing hook, unchanged.
- The platform admin: no `member` row in it. They reach it later through the admin area (feature 23).
- On first sign-in the existing session hook makes it the active business, because the client belongs to exactly one.

A setup is never half made (step 3b.1's review, F-35 and F-38; the claims replace the advisory locks after step 3b.3's review, F-49)

- One setup per email and per address at a time, through `client_setup_claim : key` (text, primary key: `email:<address>` or `slug:<address>`), `claimId` (text, a random id per setup), `claimedAt` (timestampz, default now). A setup first deletes claims older than five minutes (left by a setup that died), then inserts both keys in one statement with `on conflict do nothing` . Short of both, it answers `409 setup_in_progress` ("This client is already being set up. Try again in a moment.") and makes nothing. It removes only its own claims when it ends, whatever happened; a failed removal is logged and the claims expire. No database connection is held while a setup runs: holding one (session or transaction advisory locks) let postgres.js 3.4.9 crash the API when that connection dropped.
- On a failure after Better Auth saved the business, the business is removed when nobody but this client is in it, then the login this setup made, and only while it belongs to no business. A login reused from an unfinished setup is kept for the next try. `409 slug_taken` is answered only when a business with someone else in it holds the address.
- The limit: if that clean-up itself fails (the database failing twice in one setup), the half-made setup stays, logged, and a retry is `email_taken` . It is removed by hand until the admin area (feature 23) can delete a client. The setup's own error is the one reported, never the clean-up's.

The server-side bypass, written down where it is used

`auth.api.createOrganization` called with a `userId` and no headers skips `allowUserToCreateOrganization` (`crud-org.mjs` , `isSystemAction`), and `auth.api.createUser` with no headers skips the admin plugin's permission check. Both are Better Auth's sanctioned server paths. In this codebase they are called only from `provision-client.ts` , which only `POST /admin/clients` calls, behind `requirePlatformAdminMiddleware` . A later server-side caller would create a business with no check at all, so any new caller needs its own gate and its own line in this contract.

Closed over HTTP after 3b.2

- `POST /api/auth/organization/create` : refused for everyone, `403` .
- `POST /api/auth/organization/invite-member` (and accepting one): refused, no role holds an invitation permission.

Frontend

- `/admin/clients/new` , platform admin only on screen; the API is the real gate. States: checking the session; signed out (to `/sign-in`); not the platform admin (a card saying this page is for the agency, with a link home); the form; sending (button disabled, "Setting up..."); field errors; a form-level error

(API down or a 500); success (business name, the client's email, the sign-in address, "No email is sent yet: tell the client yourself", and "Set up another").

- Whether the viewer is the platform admin, for showing links and cards only, comes from Better Auth's session in the browser (`authClient.getSession` , `user.role`). It never grants anything.
- Every name and email is rendered as React text, never as HTML.

Testing

- Shared: the schema accepts a normal entry; trims and lowercases the email; rejects an empty or too long client name, a bad email, a blank business name.
- Backend (`admin-routes.test.ts` , real local Postgres, the existing local-dev-database guard): every refusal, the happy path's rows, the two conflicts, the forced failure leaving no login, the new client's sign-in and `/me` . 3b.2 adds the two closed doors.
- Each new test is shown able to fail once (the check removed or the value changed, run red, restored), as earlier steps did.
- Frontend: no test runner exists; the build, the lint and the browser check in 3b.3's Done when. No Browser tests command exists, so none is added.

Notes for the AI

- Better Auth 1.7.5's create paths were read from the installed source (`plugins/admin/routes.mjs` `createUser` , `plugins/organization/routes/crud-org.mjs` `createOrganization`), not its docs. `addMember` has no HTTP route, so it is server-only already; this feature does not need it, because creating the business with the client's `userId` makes them its member.
- Better Auth's two creates do not share a transaction with each other, so the compensation in step 6 of `provision-client.ts` is the atomicity boundary. Name it out loud: a hand-written single transaction (as the seed does) was passed over because it would bypass Better Auth's own create path and its hooks.
- Never pass `c.req.raw.headers` into the two `auth.api` calls.
- The session hook and `getOrganizationFromSession` need no change.
- The platform admin's dashboard still works with their dev business; with none, they now land on `/admin/clients/new` instead of a refused form.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `zod-validation/admin-validation-schemas/provision-client-validation-schema.ts` : the one rule for setting up a client, used by the form and by `POST /admin/clients` : `businessName` , `clientName` (1 to 100) and `clientEmail` . It reuses two field rules that now have files of their own: `organization-validation-schemas/business-name-validation-schema.ts` (2 to 80 characters, and refused when `toSlug` would make an empty address, so "!!" is caught under its field) and `auth-validation-schemas/email-address-validation-schema.ts` (trimmed and lowercased, as Better Auth stores it), which sign-in's schema also uses. The old `createOrganizationValidationSchema` went with its page.
- `db/admin-tables/client-setup-claim-table.ts` and `migrations/0005_client_setup_claim.sql` : `client_setup_claim` (`key` text primary key, `claimId` , `claimedAt`). A setup in progress claims `email:<address>` and `slug:<address>` . Applied to the local database only; Railway gets it at deploy.

backend

- `lib/admin/provision-client.ts` is the feature. In order: the address from `toSlug` ; claims older than five minutes cleared; both keys claimed in one insert with `on conflict do nothing` , or `setup_in_progress` ; the email checked (a login that belongs to a business, or is the platform admin's, is `email_taken` ; a login with no business is an unfinished setup and is reused, its name replaced by the one typed); the address checked; then Better Auth's `createUser` and `createOrganization` , both with no request headers, so the client is the creator and so the owner, and the existing `afterCreateOrganization` hook makes the first person linked to them. If the business fails, `removeBusinessMadeFor` removes a business at that address that has no member but this client, then the login this setup made is deleted only while it belongs to no business. Every unexpected error is logged through `safeErrorReason` and rethrown as a plain error, because Hono logs what reaches it and a database error names the email. The claims are released in `finally` , only this setup's own.
- Why claims and not a lock: Postgres advisory locks (session, then transaction) both hold a connection for the whole setup, and postgres.js 3.4.9, its newest release, throws an uncaught `TypeError` from `connection.js` when a held connection drops, which would take the API down. Frank chose the claim table over a lock in the API's memory, which would be silently wrong with two API copies (F-49).
- `middleware/auth-middleware/require-platform-admin-middleware.ts` : 401 without a session, 403 for anyone whose `user.role` is not `admin` . Not tied to a business.
- `routes/admin-routes.ts` : `POST /admin/clients` , the gate, then Hono's `validator("json")` with the shared schema (it types the body for `AppType` ; broken JSON gets Hono's own plain 400). Refusals: `bad_request` , `email_taken` , `slug_taken` , `setup_in_progress` , all added to `lib/errors/refuse.ts` . `app.ts` mounts the dashboard CORS, origin check and no-store on `/admin/*` .

- `lib/auth/auth-server.ts` : `allowUserToCreateOrganization: false` for everyone, the platform admin included (Better Auth's create would make the presser the owner), and `invitation: []` for owners and business admins. The comments now say the platform admin can set `user.role` through the admin plugin.
- `lib/errors/safe-error-reason.ts` moved from `lib/calendar/` , now that the admin code uses it too.
- `routes/admin-routes.test.ts` : 25 tests through the real app against the local database: every refusal, the rows a setup leaves, the client signing in to their own business, unfinished setups finished, failures before and after Better Auth's save cleaned up, the double click with the first setup held mid-way, claims held by another setup and expired claims, 14 setups at once, no email in a log after a real database error, the `/admin` guards, and Better Auth's own create and invite refused.

frontend

- `app/admin/clients/new/page.tsx` checks the session in the browser: signed out goes to sign-in, anyone but the platform admin sees "This page is for the agency", the platform admin gets the form. The API is the real gate.
- `components/admin/new-client-form.tsx` : the first form on the Forms standard, shadcn `Input` and `Label` (`components/ui/` , written from the shadcn preview so no install ran) with react-hook-form's `Controller` and the shared rule through `zodResolver` . A taken email or name comes back under its own field with the cursor moved there; anything else shows at the bottom. The success card says what to tell the client, because no email is sent until feature 6. Both carry Sign out, and "Back to the dashboard" when the platform admin has a business.
- `lib/api-client.ts` : `provisionClient()` through the dashboard client, one result per screen. `lib/is-platform-admin.ts` reads the role for display only.
- `app/page.tsx` : "Set up a client" on the dashboard for the platform admin; a login with no business sees "Your login has no business yet" with Sign out, and the platform admin with none is sent to the form. `SignOutLink` moved to `components/sign-out-link.tsx` so the admin page can use it. The old `/create-organization` page is deleted, and `Field` lost its unused `hint` .

Not done here, on purpose

- No welcome email: feature 6 brings email. Production sign-in cannot send a code until then either.
- Listing, editing or removing clients, and an audit trail: feature 23.
- Before the first client-facing deploy, check that the live `invitation` table is empty (F-47), and apply migration 0005 to Railway.
- The overview's plan fingerprint was left as it was: it differs from the plans because of earlier plan edits, not this feature, and rewriting it would hide that drift. `/overview` refreshes it.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-12, F-13 and F-16 were found in feature 1 and resolved here; F-14, F-32, F-34, F-47 and F-52 stay open in the live ledger.

3b/F-12 [P2] closed - Comments still describe open signup, self-serve business creation and server-side validation the code does not have

File: frontend/app/sign-in/page.tsx:17 **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** The F-05 repair changed the security model and updated the comments beside the two lines it touched, but not the ones that describe the same model elsewhere. This project treats its comments as the record of why (`coding-standards.md` , Comments), so a comment stating the model backwards is a wrong instruction to the next item, and items 3b and 25 are the ones that will read these files to change exactly this behaviour.

- `sign-in/page.tsx:17-20` says the emailOTP plugin "creates the account on first successful code, so this one screen is both sign-in and sign-up". The opposite is true since `disableSignUp: true` (`backend/src/lib/auth.ts:232`), and the same file says so correctly at lines 63-71.
- `sign-in/page.tsx:107-109` , `app/page.tsx:129-131` and `create-organization/page.tsx:13` describe a user with no business being sent to create their first one. Only the platform admin may create a business (`auth.ts:174-175`); an ordinary user is refused there (see F-13).
- `sign-in/page.tsx:43-44` ("the same schema the API validates against") and `packages/shared/src/validation/auth.ts:6-7` ("the API validates the same values a second time") claim server-side use of the shared schemas. Nothing under `backend/src` imports `@scheduleads-app/shared/validation` ; Better Auth validates with its own rules (`z.email()` in `email-otp/routes.mjs:95` , `min(1)` for an organization name in `crud-org.mjs`).
- `backend/src/lib/auth.ts:184-190` says a `plan` sent on organization update "throws". Read off better-auth 1.7.5, it is silently dropped instead: `toZodSchema` omits `input: false` fields client-side (`db/to-zod.mjs:7`), the `data` object strips unknown keys, and better-call replaces the body with the parsed value (`better-call/dist/validator.mjs:17`). Still safe, but the comment tells a reader that an update returning 200 would have been refused. Not observed live, since it needs a signed-in owner.

Suggested fix: Rewrite those comment lines to match the code: sign-in only, no self-serve business creation, the shared schemas used by the forms only until a route owned by this API validates with them, and `plan` silently discarded on both create and update. **Resolution:** Step 3b.2, 2026-09-30: the backend and shared lines are corrected (`auth-server.ts` sign-up and hook comments, `sign-in/page.tsx:36` , which no longer claims the API uses the same schema). The `plan` comment already matched the code. Left for step 3b.3: `app/page.tsx:98` ("go create one") and the create page, which 3b.3 deletes. Repaired by step 3b.3, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.3's review to close. The last line, `app/page.tsx` 's "go create one", now describes the no-business card and the platform admin's way to set up a client; the create page it pointed at is deleted. No comment in the touched files describes open signup or self-serve business creation. Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-

examining a67b75b): `app/page.tsx:100-102` now describes the no-business card and the platform admin's way to set up a client, `/create-organization` is gone, `sign-in/page.tsx:2` and `:36` say sign-in only and that Better Auth checks the address with its own rule, and the shared schema comments claim server use only where it is true (`provision-client-validation-schema.ts:2` , which `admin-routes.ts:25` does parse). The `plan` comment (`auth-server.ts:128-130`) says a sent plan is silently dropped. A search of the touched frontend, backend and shared files finds no comment left describing signup, self-serve creation or shared server-side validation.

3b/F-13 [P3] closed - A signed-in user with no business is sent to a create form that can only refuse them, with no way to sign out

File: `frontend/app/page.tsx:155` **Found:** 2026-09-23 by /audit (scope: current; lens: quality) **Why it matters:** `PickOrganization` sends any user whose organization list is empty to `/create-organization` (`page.tsx:155-157`). Since the F-05 repair only a platform admin can create one, so every ordinary user who lands there gets Better Auth's "You are not allowed to create a new organization" on submit, and `create-organization/page.tsx` renders no sign-out control, unlike every other signed-in state. Reachable today through the provisioning path the spec itself names: until item 3b, "a new user row is a manual database act" (`auth.ts:227-230`), so a user created before their membership, or one whose only business is deleted, lands on a dead end. Read off the code, not observed live; Frank's walkthrough saw the refusal message itself for `owner@example.com` .

Suggested fix: When the list is empty and the user is not a platform admin, render an `AuthCard` saying the account has no business yet and to contact the agency, with `SignOutLink` . Add a sign-out control to the create page as well. **Resolution:** Repaired by step 3b.3, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.3's review to close. `PickOrganization` (`app/page.tsx`) shows an ordinary login with no business the card "Your login has no business yet" with Sign out, and sends the platform admin to `/admin/clients/new` ; `/create-organization` is deleted. Checked in the browser against the local API: a test client whose business was removed saw the card and signed out. Not closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining a67b75b): the ordinary login's half is gone (`app/page.tsx:163-173` , the card with `SignOutLink`). The other half is not: a platform admin with no business is now sent to `/admin/clients/new` , which, like the create page before it, has no Sign out (F-48). This finding's suggested fix asked for a sign-out on that page as well, so it stays `fixed` and closes with F-48. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): both halves are gone at `42315d9` . An ordinary login with an empty list gets the "Your login has no business yet" card with `SignOutLink` (`app/page.tsx:164-174`); the platform admin is sent to `/admin/clients/new` (`app/page.tsx:126-129`), whose form and success card now carry `AdminFooter` with Sign out (`new-client-form.tsx:115-126` , `:156` , `:169` , see F-48). `/create-organization` no longer exists and nothing links to it.

3b/F-16 [P3] closed - Better Auth endpoints already open two paths the spec reserves for later items

File: backend/src/lib/auth.ts:85 **Found:** 2026-09-23 by /audit (scope: current; lens: security) **Why it matters:** Neither is a breach, since signup is closed and both need a consenting existing user or an existing platform admin, but each contradicts a written contract and is live over HTTP today.

- The `owner` role is granted `invitation: ["create", "cancel"]` (`auth.ts:88`), so `/organization/invite-member` and `/organization/accept-invitation` (`organization/routes/crud-invites.mjs:42`, `:246`) work now. An owner can invite any existing user, and if they accept they join a second business. The spec's Out of scope reserves "any path at all that puts a client user inside a business" for item 3b, and a second membership is what drives a user into the pick-a-business state.
- `admin()` exposes `/admin/set-role` and `/admin/create-user` (`admin/routes.mjs:43`, `:133`) to any platform admin. The spec's data contract says no request path may write `user.role` before item 23, and `auth.ts:131-132` and `schema.ts:52-55` say promotion is a manual database edit and nowhere else.

Suggested fix: Give `owner` and `admin` an empty `invitation` list until item 3b decides the invitation flow, and correct the `user.role` contract and comments to say a platform admin can set it through the admin plugin's endpoint, or record either as accepted with the reason. **Resolution:** Repaired by step 3b.2, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.2's review to close. `owner` and `admin` now hold `invitation: []` (`auth-server.ts`), and a route test shows an owner's `/organization/invite-member` is 403 with no invitation row; the `user.role` comments (`auth-server.ts`, `user-table.ts`) now say the platform admin can set it through the admin plugin. Shown able to fail: with `invitation: ["create", "cancel"]` back, the invite test failed. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining ea23e45): `owner`, `admin` and `member` all hold `invitation: []` (`auth-server.ts:62-78`), and Better Auth's `/organization/invite-member` gates on `hasPermission` (`crud-invites.mjs:95`), so the route test's 403 with no invitation row is the real refusal; `addMember` has no HTTP path (`crud-members.mjs:25`, a pathless `createAuthEndpoint`). The `user.role` comments (`auth-server.ts:92-94`, `user-table.ts:15-16`) now match the admin plugin (`createUser` and `set-role` gate on `user:set-role`, `admin/routes.mjs`) and `input: false` (`db/schema.mjs:65-74`, a 400). Accepting an invitation made before this change is not role-gated; see F-47.

3b/F-35 [P2] closed - A failure after Better Auth has saved the business leaves a business with no owner, and is reported as a taken name

File: backend/lib/admin/provision-client.ts:54 **Found:** 2026-09-30 by /audit (scope: step 3b.1; lens: quality, security) **Why it matters:** Better Auth 1.7.5's `createOrganization` saves the business, then its owner row, then runs `afterCreateOrganization` (the first person), as separate writes with no transaction around them (`crud-org.mjs`, the org adapter's `createOrganization` and `createMember`). If anything after the first write throws, the catch at `provision-client.ts:54-60` removes the new login (the owner row goes with it, by cascade) but leaves the business saved, with no owner and no person. It then sees the slug as taken and answers `409 slug_taken`, "A business with that name already exists", hiding a real fault behind a wrong message, and every retry with that name is refused for good. On the

reuse path the login is kept, so the business keeps its owner row but has no first person, and a retry is `email_taken`. The spec's promise is "never half made"; this half is the business, not the login. Needs a database fault mid-request, so not P1. **Suggested fix:** In the catch, before deciding `slug_taken`, remove a business at that slug that this request made: one whose only members are this client, or which has no members at all, and was not there at the pre-check. Then answer `slug_taken` only when a business at that slug still exists after that, and rethrow otherwise. Add a test that forces the failure after the business is saved (for example a failing first-person insert) and asserts no business, no new login and a 500.

Resolution: Fixed 2026-09-30 on Frank's yes, in the step 3b.1 review-fix commit. On a failure, `removeBusinessMadeFor` (`provision-client.ts`) removes a business at the slug that has no member but this client (its owner row and first person go by cascade), then the login this setup made; `slug_taken` is answered only when a business with someone else in it holds the slug. Two route tests force a failure right after Better Auth's save: a new login (no business, no login left, 500) and a reused one (no business, login kept, the retry answers 201). Shown able to fail by removing the business delete: both failed. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining 76ef009 and ea23e45): Better Auth 1.7.5 saves the business, the owner row and then runs `afterCreateOrganization` as separate writes (`crud-org.mjs:74`, `:100`, `:137`); a failure after any of them now reaches `removeBusinessMadeFor` (`provision-client.ts:90`), which removes a business at the slug with no member but this client (zero members included, so a failed owner insert is covered too) and answers `slug_taken` only when someone else is in it. The slug was checked free under its lock at `:64`, so the business removed can only be this setup's. Both route tests pass (backend suite 119/119). Not introduced by the repair but worth knowing: if the clean-up itself fails the half setup stays, see F-44.

3b/F-36 [P3] closed - The spec still says broken JSON is answered with the refusal shape

File: blueprint/context/current-feature.md:259 **Found:** 2026-09-30 by /audit (scope: step 3b.1; lens: quality) **Why it matters:** Data / contracts lists "malformed JSON" under `400 bad_request`. The route checks the body with Hono's `validator("json")`, which throws its own `HTTPException(400, "Malformed JSON in request body")` before the schema callback runs (`hono/dist/validator/validator.js`), so broken JSON gets a plain-text 400. The build log records the departure; the spec, which later steps and `/complete` read, does not. **Suggested fix:** Correct the contract line: malformed JSON is Hono's own plain 400; a missing or wrong body is `400 bad_request`. **Resolution:** Carried to step 3b.2 on Frank's call, 2026-09-30. Written into that step's plan in the spec. Repaired by step 3b.2, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.2's review to close. The spec's Data / contracts and step 3b.1's route line now say malformed JSON is Hono's own plain-text 400 and a missing or wrong body is `400 bad_request`. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining ea23e45): the contract now matches hono 4.13.8 `validator/validator.js:14-21` (a JSON content type with an unparseable body throws `HTTPException(400, "Malformed JSON in request body")`; no JSON content type leaves `{}`, which the schema refuses as `bad_request`). One nuance, not worth reopening: an empty body sent as `application/json` is also Hono's plain 400, since `c.req.json()` fails on it.

3b/F-37 [P3] closed - No test proves the /admin routes carry the dashboard's guards

File: backend/routes/admin-routes.test.ts:120 **Found:** 2026-09-30 by /audit (scope: step 3b.1; lens: tests) **Why it matters:** `app.ts` mounts the dashboard CORS, origin check and `no-store` on `/admin/`, and the calendar tests prove the same for `/calendar` with a cross-site form post. Nothing in `admin-routes.test.ts` would fail if the `/admin/` line were dropped, and an answer naming a client's login is exactly what `no-store` exists to keep out of caches. **Suggested fix:** Assert `Cache-Control: no-store` on one answer, and that a form-encoded post from another origin with the platform admin's cookie is refused (403) and makes nothing. **Resolution:** Carried to step 3b.2 on Frank's call, 2026-09-30. Written into that step's plan in the spec. Repaired by step 3b.2, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.2's review to close. Two route tests: a 201 answer carries `Cache-Control: no-store`, and a form-encoded post from another origin with the platform admin's cookie is 403 and makes no login. Shown able to fail: without the `/admin/*` guard line in `app.ts`, both failed. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining ea23e45): `admin-routes.test.ts:395-415` asserts `Cache-Control: no-store` on a 201 and a 403 with no login for a cross-origin form post carrying the platform admin's cookie. The 403 can only come from the origin check: without `app.ts:22` the form body skips Hono's JSON validator (`validator.js:14`, non-JSON content type leaves `{}`) and the schema answers 400, so the test is not satisfied by any other guard.

3b/F-38 [P2] closed - Two overlapping setups for one email can give the client two businesses, or delete the owner of a business reported as made

File: backend/lib/admin/provision-client.ts:57 **Found:** 2026-09-30 by /audit independent (scope: step 3b.1; lens: quality, security, tests) **Why it matters:** A setup still in flight looks exactly like an unfinished one: between `createUser` (line 41) and the business being saved, the new login has no membership, so a second request for the same email takes the reuse path at line 29. Nothing serializes the two. Reproduced against the local database with a throwaway probe (removed after; no product file changed), holding the first request's `createOrganization` 400 ms to widen the window:

- same email, two business names: both answered `201` and the client ended with two memberships. That is option B, which Frank refused on 2026-09-30 ("one login is one business"), and the session hook then pre-selects nothing.
- same email, same business (a double submit or a retried request): the second request reused the login and made the business; the first lost the slug race, and its catch deleted the login it had made (line 57) without checking it now owns a business. Result: `201` naming a client id that no longer exists, a business with no member and no login, and `409 slug_taken` on the other. Without the delay, two simultaneous identical requests gave `500` and `201` (the second `createUser` refused the duplicate). Only one platform admin uses this route and the form will disable its button, so the window is narrow, but the outcome is silent and the 201 is wrong. No test covers overlapping requests.

Suggested fix: Serialize setups per email, for example a Postgres advisory lock on the email's hash held across the two creates (or a reuse rule that also requires the login to be older than a setup can take). Independently, make the compensation delete the login only while it still has no membership, so it can never remove another request's owner. Add a test that runs two overlapping setups for one email and asserts one business, one owner, and no 201 for a deleted client. **Resolution:** Fixed 2026-

09-30 on Frank's yes, in the step 3b.1 review-fix commit. `withSetupLocks` holds two Postgres advisory locks on one reserved connection for the whole setup, email first then address, so setups for one email (or one address) run one after another and the second sees the first's finished login as `email_taken`. The login clean-up also deletes only while the login belongs to no business (`removeLoginWithoutBusiness`); with the locks no request can reach that case, so no test exercises that guard. Two route tests hold the first setup 300 ms and start a second for the same email 50 ms later, with the same business and with another: one 201, one 409, one business, one owner. Shown able to fail by removing the locks: both failed. Waits for the next review to close. Not closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining 76ef009 and ea23e45): the original race is gone (the two overlap tests pass, and setups for one email now run one after another), but the repair introduced a new defect: each setup holds one of the pool's ten connections while it waits for, and then holds, its locks, so ten setups at once leave none for their own queries and the whole API stops answering (F-43, reproduced). F-42 is also a defect of this repair. Stays `fixed` until F-43 is resolved. Not closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b and a67b75b): F-43 is now closed and both overlap tests pass (backend 120/120), but the lock design this repair introduced has a further defect: a lock connection that drops mid-setup crashes the API process or hangs the setup, because the unlock is sent on a dead reserved connection (F-49, reproduced). Stays `fixed` until F-49 is resolved. Re-repaired 2026-09-30 in the step 3b.3 review-fix commit: the race is now stopped by `client_setup_claim` (see F-49), not advisory locks. The second of two overlapping setups answers `409 setup_in_progress` and makes nothing; the clean-up still deletes a login only while it belongs to no business. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): `withSetupClaims` (`provision-client.ts:47-72`) inserts `email:<address>` and `slug:<address>` in one statement against the `client_setup_claim` primary key with `on conflict do nothing`, and runs the setup only when both rows came back, so no second setup for the same email or address can reach `findLoginForEmail` while the first is between its two creates. `removeLoginWithoutBusiness` (`:197-212`) still deletes only while the login has no membership. Both overlap tests (`admin-routes.test.ts:347-389`) pass with the second answering `setup_in_progress`, then `email_taken` once the first's claims are gone (backend 122/122). The repair holds no connection, so it carries neither F-43 nor F-49. Known limit, written in the spec: a setup still running after five minutes can have its claims cleared by the next one.

3b/F-39 [P3] closed - Finishing an unfinished setup ignores the client name sent with it

File: backend/lib/admin/provision-client.ts:40 **Found:** 2026-09-30 by /audit independent (scope: step 3b.1; lens: quality) **Why it matters:** On the reuse path the login is returned as stored and `input.clientName` is never used, so the answer's `client.name` is the earlier attempt's name, not the one just typed. The contract says the user is "the name given". The likeliest reason to run a setup again is correcting the first attempt, and the correction is dropped without a word. The reuse test (`admin-routes.test.ts`) sends "Client finished" against a login named "Unfinished" and does not assert the name either way. **Suggested fix:** Either update the reused login's name to the one given (one `db.update(user)` before creating the business, or Better Auth's admin `updateUser` with no headers), or write in the spec that the first name stands, and assert the chosen behaviour in the reuse test.

Resolution: Carried to step 3b.2 on Frank's call, 2026-09-30. Written into that step's plan in the spec. Repaired by step 3b.2, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.2's review to close. `renameLogin` (`provision-client.ts`) writes the name typed now over the reused login's, and the answer returns it; the reuse test checks both. Shown able to fail: without the rename, the test failed. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining ea23e45): `renameLogin` (`provision-client.ts:121-125`) runs only on the reuse path, under the email lock, before the business is made, and the answer is built from its return; `admin-routes.test.ts:213-217` asserts the new name in both the answer and the stored login. A rename that survives a later failed business is intended (the next try keeps the correction).

3b/F-40 [P3] closed - Two shared field schemas live in files named after other schemas

File: packages/shared/zod-validation/organization-validation-schemas/create-organization-validation-schema.ts:6 **Found:** 2026-09-30 by /audit independent (scope: step 3b.1; lens: quality) **Why it matters:** The naming standard is one file per export, the file named after the thing imported.

`businessNameValidationSchema` now lives in `create-organization-validation-schema.ts` and `emailAddressValidationSchema` in `sign-in-email-validation-schema.ts`, so the import in `provision-client-validation-schema.ts:6-7` reads as if it reused the sign-in and create-business forms. Step 3b.3 deletes `/create-organization`, its only user, which leaves `createOrganizationValidationSchema` dead and the live business-name rule in a file named after it. The rule's message, "Enter the name of your business.", is also now shown to the platform admin about the client's business. **Suggested fix:** Give each field schema its own file, named after it, in its area folder, export them through `index.ts`, and remove `createOrganizationValidationSchema` with its page in 3b.3. Word the business name message so it reads right on both forms. **Resolution:** Carried to step 3b.3, which deletes the create page the business-name rule's file is named after on Frank's call, 2026-09-30. Written into that step's plan in the spec. Repaired by step 3b.3, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.3's review to close. `businessNameValidationSchema` lives in `business-name-validation-schema.ts` (the old create-organization file, renamed; `createOrganizationValidationSchema` is gone with its page) and `emailAddressValidationSchema` in `email-address-validation-schema.ts`; the business-name message reads "Enter the business's name." Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining a67b75b): `business-name-validation-schema.ts` and `email-address-validation-schema.ts` each hold the one export they are named after, `zod-validation/index.ts:4` and `:7` export them, `provision-client-validation-schema.ts:6-7` and `sign-in-email-validation-schema.ts:5` import them from their own files, and `createOrganizationValidationSchema` has no remaining reference. The message reads for either form. Shared tests 55/55, both builds pass.

3b/F-41 [P3] closed - A database error inside a setup may print the client's email in the API's log

File: backend/lib/admin/provision-client.ts:61 **Found:** 2026-09-30 by /audit independent (scope: step 3b.1; lens: security), raised as a remaining risk and recorded by the builder **Why it matters:** The spec says

the client's email never goes in a log line. `provisionClient` rethrows unexpected errors (`:61`), and Hono's default error handler logs them. A drizzle query error carries its parameters, so a failure inside `createUser` (a duplicate email in the F-38 race, for example) would print the email. Not observed: no run produced such a log line. **Suggested fix:** Confirm by forcing a failing `createUser` and reading the log. If it shows, log a safe reason (the project already has `safe-error-reason.ts` for the calendar) and rethrow a plain error without the query's parameters. **Resolution:** Carried to step 3b.2 on Frank's call, 2026-09-30. Written into that step's plan in the spec. Confirmed 2026-09-30 by a route test before any repair: a real duplicate-key error inside the login's creation reached Hono's error log as `Failed query: insert into "user" ...` with the client's email in its values. Repaired by step 3b.2, 2026-09-30, and set to `fixed` by the builder; waits for step 3b.2's review to close. `provisionClient` now logs `[admin] a client setup failed: <safe reason>` (the database error reduced to its Postgres code by `safeErrorReason`, moved from `lib/calendar` to `lib/errors` now that two areas use it) and rethrows a plain error. The same test now passes; with the raw error rethrown it fails again. Closed 2026-09-30 by /audit independent (scope: step 3b.2, re-examining ea23e45): re-shown by planting `throw error` back in `provisionClient`'s catch (the F-41 test failed, 21 of 22 passed) and restoring it byte for byte. Every error out of the locked setup passes through that one catch (`provision-client.ts:30-37`), drizzle 0.45.2's `DrizzleQueryError` message starts with "Failed query" (`errors.js:12`) so `safeErrorReason` keeps only its Postgres code, and Better Auth logs nothing itself on a server-side `auth.api` call (its router `onError`, `api/index.mjs:212`, runs only for HTTP).

3b/F-42 [P3] closed - If releasing the setup locks fails, the reserved connection is never handed back and the real error is lost

File: backend/lib/admin/provision-client.ts:52 **Found:** 2026-09-30 by /audit (scope: step 3b.2 and step 3b.1's fixes; lens: quality, performance) **Why it matters:** `withSetupLocks` runs `pg_advisory_unlock_all()` and then `connection.release()` in one `finally`. If the unlock query throws (the connection dropped mid-setup), `release()` never runs, so that reserved connection may never return to the pool of ten, and the unlock error replaces whatever error the setup itself threw. Needs a dropped connection, so rare. **Suggested fix:** Release in its own `finally` (or `try { unlock } finally { release }`), and do not let an unlock failure replace the setup's own error; a dropped connection already frees its advisory locks. **Resolution:** Independent review 2026-09-30 (/audit independent, scope: step 3b.2): agreed, P3, with one correction. A dropped connection does go back: postgres.js 3.4.9's `onclose` (`cjs/src/index.js:421-427`) moves it to the closed list and clears `reserved`, so the pool reopens it. What stands is the error masking, and one case worse than described: when the setup itself succeeded, a failing unlock turns its 201 into a 500, and the retry the platform admin then makes is `email_taken`. Release and unlock should never be able to replace the setup's own result. Fixed 2026-09-30 on Frank's yes, in the step 3b.2 review-fix commit; waits for step 3b.3's review to close. `releaseSetupLocks` runs the unlock in its own try, logs a failure through `safeErrorReason` without throwing, and always releases the connection in `finally`, so an unlock failure can neither replace the setup's own error nor turn a finished setup into a 500. No test forces an unlock failure; checked by reading. Not closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b): the

repair is right for an unlock that fails with an error (logged, the connection always released, the setup's own result kept). But the case this finding names, a connection dropped mid-setup, never reaches that `catch`: postgres.js throws the unlock's failure outside the promise, so the process crashes or the unlock hangs (F-49, reproduced), and the new comment at `provision-client.ts:58` says the opposite. Stays `fixed` and closes with F-49. Superseded 2026-09-30 by the F-49 repair: there is no unlock any more. `releaseSetupClaims` deletes this setup's own claims, logs a failure through `safeErrorReason` and never throws, so it cannot replace the setup's own outcome; a claim left behind expires after five minutes. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): there is no reserved connection and no unlock left (no `reserve()`, `begin()` or advisory call anywhere in `backend` or `packages/shared`; `database.ts` is unchanged from `main`). `releaseSetupClaims` (`provision-client.ts:76-82`) runs in the `finally` of `withSetupClaims`, catches its own failure, logs only `safeErrorReason`, and returns nothing, so neither a finished setup's 201 nor a failed setup's own error can be replaced by it.

3b/F-43 [P2] closed - Ten setups at once take every pooled connection and freeze the whole API

File: backend/lib/admin/provision-client.ts:45 **Found:** 2026-09-30 by /audit independent (scope: step 3b.2; lens: performance) **Why it matters:** `withSetupLocks` reserves one of the pool's ten connections (`database.ts:19`, `max: 10`) for the whole setup, including while it waits on `pg_advisory_lock`, but the setup's own queries and Better Auth's run on other pool connections. With ten setups in flight (any emails), all ten connections are reserved, every setup's next query waits for a free one, none is ever released, and every other request in the API (sign-in, `/me`, the public booking pages of every tenant) waits behind them until the process restarts. Nothing times out. Reproduced against the local database with a throwaway probe calling `provisionClient` ten times at once: nothing finished in 8 s and a plain `select` on `db` then hung too; with nine, all nine answered 201. The probe was removed and left no rows or advisory locks. Only the platform admin can call the route and the form will disable its button, so it needs a script, a retry loop or a buggy client, but the cost is a full outage for every tenant, and the same holds for setups queued on one email, each holding a connection while it waits. **Suggested fix:** Keep the lock connection out of the query pool: a separate small `postgres()` client used only for the setup locks, or a cap on setups in flight below the pool size (one at a time is plenty for one platform admin), with a bounded wait (`pg_try_advisory_lock` in a short retry loop, or `lock_timeout` on the lock connection) so a stuck setup cannot queue others forever. Add a test that starts more setups at once than the pool holds and asserts they all answer. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.2 review-fix commit; waits for step 3b.3's review to close. The locks now take their connection from `advisoryLockClient` (`database.ts`, two connections of its own), never the API's pool of ten, so waiting setups queue there. A route test runs 14 setups at once: all 201, and `/health` still answers. Shown able to fail: with the lock back on `db.$client.reserve()`, that test timed out at 15 s and every test after it hung, the freeze reproduced. Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b): `withSetupLocks` reserves from `advisoryLockClient` (`database.ts:27`, `max: 2`) while every query the setup and Better Auth run uses `db`'s pool of ten, so waiting setups queue in

postgres.js's `reserve()` (`index.js:203-211` , a promise on the lock pool's own queue) and never hold an API connection. Email locks are always taken before address locks, so the two lock connections cannot deadlock each other. The 14-setup test passes (backend 120/120) and asserts `/health` afterwards. The suggested bounded wait was not added: a setup stuck inside Better Auth blocks later setups, but no longer the API, which is what the spec's contract promises. The lock pool does carry F-49 (a dropped lock connection), which predates this repair.

3b/F-44 [P3] closed - A failed clean-up leaves a half-made setup that no retry can finish, and hides the real error

File: backend/lib/admin/provision-client.ts:90 **Found:** 2026-09-30 by /audit independent (scope: step 3b.2; lens: quality) **Why it matters:** The spec promises "A setup is never half made". The promise holds only while the clean-up's own queries work. The likeliest reason `createOrganization` fails after saving the business is a database fault, and then `removeBusinessMadeFor` (unguarded) usually fails too: its error replaces the original one in the log, and the business stays with the client as its owner and possibly no first person. Every retry is then `409 email_taken` , because the login now belongs to a business, and nothing in the product can remove it before feature 23. Not introduced by the F-35 repair (the earlier code ended the same way under a database fault), and it needs a database fault mid-setup, so P3. **Suggested fix:** Write the limit into the spec's "A setup is never half made" contract (it holds unless the database fails during the clean-up; the fix is then by hand), and log the setup's own safe reason before the clean-up runs, so a failing clean-up cannot hide it. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.2 review-fix commit; waits for step 3b.3's review to close. The clean-up runs in its own try: a failure there is logged with a safe reason and the setup's own error is the one rethrown. The spec's "A setup is never half made" now states the limit: if the clean-up itself fails, the half-made setup stays, a retry is `email_taken` , and it is removed by hand until feature 23. No test forces a failing clean-up; checked by reading. Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b): the clean-up (`provision-client.ts:104-113`) runs in its own try, logs only `safeErrorReason` , and the setup's own error is rethrown at `:116` , then logged safely and replaced by a plain error at `:35-36` , so neither the clean-up's error nor a query's parameters reach Hono. When the business delete fails, the new login's delete is skipped, which leaves a login with no business that the next setup with that email finishes, as intended. The spec's "A setup is never half made" states the limit. Checked by reading, as the builder did.

3b/F-45 [P3] closed - The overlap tests start the second setup on a fixed 50 ms timer, not once the first holds the lock

File: backend/routes/admin-routes.test.ts:331 **Found:** 2026-09-30 by /audit independent (scope: step 3b.2; lens: tests) **Why it matters:** Both overlap tests assume the first request has taken its locks within 50 ms (session lookup, a connection reserved, two lock queries, two reads, `createUser`). If it has not, on a cold pool or a busy machine, the second request takes the locks first, the 300 ms hold (a `mockImplementationOnce`) lands on it instead, and the first answers 409: the tests then fail on `first.status` although the code is right. They pass today on the laptop. **Suggested fix:** Start the second setup from inside the mocked `createOrganization` (resolve a promise when the mock is entered, then

wait on it), so it always overlaps a first setup that already holds both locks. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.2 review-fix commit; waits for step 3b.3's review to close. The overlap tests now send the second setup only once the first is inside its locks (the held `createOrganization` signals a promise the test awaits), with no fixed timer. Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b): `holdFirstSetup` (`admin-routes.test.ts:318-331`) resolves `inside` from within the mocked `createOrganization`, which runs only after both locks are held, and the second request is sent after `await held.inside` (`:334-339`, `:356-361`). The mock is `Once`, so the second setup gets the real create. Both tests pass; no timer decides who takes the lock first.

3b/F-46 [P3] closed - The new tests carry finding and step numbers in their comments and names

File: `backend/routes/admin-routes.test.ts:269` **Found:** 2026-09-30 by /audit independent (scope: step 3b.2; lens: quality) **Why it matters:** `coding-standards.md` (Comments, "The balance") keeps history out of code, naming step and finding numbers. This delta adds `// F-35: (:269)`, `// F-38: (:315)`, `// F-41: (:369)` and the describe names "(F-37)" (`:395`) and "(step 3b.2)" (`:417`). The ledger is archived and reset by `/complete`, so these numbers point at nothing a reader can find later. **Suggested fix:** Keep the plain-words part of each comment and name, drop the numbers ("the old doors are closed", "the /admin routes keep the dashboard's guards"). **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.2 review-fix commit; waits for step 3b.3's review to close. No finding or step number is left in `admin-routes.test.ts` or `provision-client.ts`; the two describe blocks are named for what they prove. Closed 2026-09-30 by /audit independent (scope: step 3b.3, re-examining 22a411b and a67b75b): a search for `F-NN`, `step N`, `3b.N` and `feature N` in `admin-routes.test.ts`, `provision-client.ts`, `database.ts` and the step 3b.3 frontend and shared files finds none in those two files; the describe blocks read "the /admin routes keep the dashboard's guards" and "Better Auth's own ways into a business are closed". The only hit is a forward pointer, "(feature 6)", in `new-client-form.tsx:121`, which names a plan item, not history.

3b/F-48 [P3] closed - The Set up a client page has no Sign out and no way back, so a platform admin with no business cannot sign out

File: `frontend/app/admin/clients/new/page.tsx:63` **Found:** 2026-09-30 by /audit (scope: step 3b.3; lens: quality) **Why it matters:** The form and its success card render no Sign out and no link to the dashboard. For a platform admin with no business, `/` now sends them straight back to `/admin/clients/new` (`PickOrganization`), so no screen they can reach offers Sign out: the same "nobody stuck" gap F-13 closed for ordinary logins. In production the platform admin may well have no business of their own until feature 10. **Suggested fix:** Give the form and the success card the footer every other signed-in card has: Sign out, plus "Back to the dashboard" when the admin has a business. **Resolution:** Independent review 2026-09-30 (/audit independent, scope: step 3b.3, re-examining a67b75b): agreed, P3, confirmed by reading. `PickOrganization` (`app/page.tsx:125-127`) sends a platform admin with an empty list to `/admin/clients/new`, and neither `NewClientForm` nor `SetUpCard` renders a footer. Not a hard lock: `/sign-in` does not turn away a signed-in user, so typing that address and signing in as someone else

replaces the session, but nothing offers Sign out. This is the platform admin's half of F-13 (whose suggested fix asked for a sign-out on the page a no-business login is sent to), so F-13 stays `fixed` until this is repaired. Fixed 2026-09-30 on Frank's yes, in the step 3b.3 review-fix commit; waits for the next review (the final one at /complete) to close. The form and the success card carry a footer with Sign out (now `components/sign-out-link.tsx`, shared with the dashboard) and "Back to the dashboard" when the platform admin has a business (the page reads `organization.list()`). Checked in the browser: both links show for the dev platform admin, and Sign out from the form lands on /sign-in. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): `AdminFooter` (`new-client-form.tsx:115-126`) renders `SignOutLink` always and "Back to the dashboard" only when `hasBusiness`, on both the form (`:169`) and the success card (`:156`). `hasBusiness` comes from `organization.list()` on the page (`admin/clients/new/page.tsx:25-37`), so a platform admin with no business is never offered a link that would bounce them back. `SignOutLink` moved to `components/sign-out-link.tsx` unchanged in behaviour and the dashboard imports it from there. Frontend build and lint pass.

3b/F-49 [P2] closed - A lock connection that drops mid-setup crashes the API process, or hangs the setup for good

File: `backend/lib/admin/provision-client.ts:63` **Found:** 2026-09-30 by /audit independent (scope: step 3b.3; lens: quality, performance) **Why it matters:** `releaseSetupLocks` sends `pg_advisory_unlock_all()` on the reserved lock connection in every `finally`, and its comment says a dropped connection is harmless ("its locks are already gone"; a failed unlock "is logged, never thrown"). In postgres.js 3.4.9 it is not. When a reserved connection's socket closes, `closed()` sets `socket = null` and `onclose` moves it to the closed list (`cjs/src/connection.js:436-458`, `cjs/src/index.js:421-427`), but a query sent on the reserved handle still goes to `c.execute` (`index.js:226-230`), whose write is scheduled with `setImmediate(nextWrite)` and then calls `socket.write` on `null` (`connection.js:246-258`). That throws outside any promise: an uncaught `TypeError`, which ends the Node process, so every tenant's API goes down until Railway restarts it, and the setup never answers. Reproduced twice, both cleaned up with no rows, locks or product files left:

- a throwaway route-level probe called `provisionClient` and, inside the held `createOrganization`, ran `pg_terminate_backend` on the lock connection: one backend killed, the setup gave no answer in 5 s, and Vitest caught "Uncaught Exception TypeError: Cannot read properties of null (reading 'write')" from `nextWrite`.
- a plain postgres.js script: the same crash for a connection killed while holding its lock; for one killed while waiting on `pg_advisory_lock` (ECONNRESET on Windows, `hadError` true, so pending queries are never errored, `connection.js:453`) the unlock never resolved, so the setup would hang and keep one of the lock pool's two connections for good; two such events and every later setup waits forever. Introduced with the locks themselves (the F-38 repair), not by the F-42 or F-43 repairs, and needs Postgres to drop that one connection during a setup (a database restart, failover or network reset), so not P1. The cost when it happens is the whole API, not one request. **Suggested fix:** Do not send a query after the setup on a connection that may be gone. Transaction-scoped locks need no

unlock at all: take both locks with `pg_advisory_xact_lock` inside `advisoryLockClient.begin(...)` and run the setup within that callback; Postgres then frees both locks at commit, rollback or connection loss. The same probe with `begin` and `pg_advisory_xact_lock` rejected cleanly with `CONNECTION_CLOSED` and the pool kept working. One trade-off to write into the spec: when the drop comes after the setup finished, the commit fails and the answer is a 500 for a setup that was made (the retry then says `email_taken`). Correct the comments that say a drop is harmless, and the spec's "released when it ends or its connection dies" line if the mechanism changes. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.3 review-fix commit; waits for the next review (the final one at /complete) to close. The planned repair (transaction-scoped advisory locks) was built and proved wrong by a route test that terminated the lock's backend mid-setup: postgres.js 3.4.9 (the newest) gave up on the transaction at once and later threw the same uncaught `TypeError ... reading 'write' from connection.js nextWrite`, while the setup ran on. Any lock that holds a connection can hit it. On Frank's call (option A, 2026-09-30) the locks are replaced by `client_setup_claim` rows (migration 0005): a setup inserts its email and address keys in one statement with `on conflict do nothing`, answers 409 `setup_in_progress` short of both, removes only its own claims when it ends, and clears claims older than five minutes first. No connection is held while a setup runs, so there is nothing to drop; `advisoryLockClient` is gone. Tests: both overlap tests now expect `setup_in_progress` for the second, then `email_taken` once the first is done and its claims are gone; a claim held by another setup gives 409 and is left alone; a claim older than five minutes is cleared. Shown able to fail: claims ignored (three tests failed), stale claims never cleared (its test failed), every claim released instead of its own (its test failed). Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): every query in a setup, the claim insert and delete included, now runs on `db`'s ordinary pool (`provision-client.ts:56-64` , `:78`), and nothing reserves a connection or holds one across the setup (`advisoryLockClient` is gone; `database.ts` matches `main`), so the reserved-handle `nextWrite` crash has no path left. A dropped pool connection fails only the query on it, which reaches `provisionClient`'s catch and is logged safely. Migration `0005_client_setup_claim.sql` matches `client-setup-claim-table.ts` and the journal; the local database has the table and no claim rows after the run. The three claim tests (`admin-routes.test.ts:347-417`) pass.

3b/F-50 [P3] closed - The sign-in Field keeps a hint prop that only the deleted create page used

File: frontend/components/auth-card.tsx:40 **Found:** 2026-09-30 by /audit independent (scope: step 3b.3; lens: quality) **Why it matters:** Step 3b.3 says to delete "the now unused pieces of `auth-card.tsx`" with `/create-organization`. The create page was the only caller that passed `hint` (its "Your address will be ..." line); the two remaining `Field`s in `sign-in/page.tsx` pass none. The `hint` prop, its id, its `aria-describedby` entry and its paragraph are now dead code, against "No unused imports or variables" and the step's own instruction. **Suggested fix:** Remove `hint` and `hintId` from `Field` and keep only the error in `aria-describedby`, or leave it with the next form that needs it. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.3 review-fix commit; waits for the next review (the final one at /complete) to

close. `Field` no longer takes `hint`; its error is linked to the input directly. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): `auth-card.tsx` has no `hint` or `hintId` left, `aria-describedby` is the error id only when an error shows, and the three remaining exports (`AuthCard`, `Field`, `Notice`) each have live callers (`Field` in `sign-in/page.tsx`). Lint passes with no unused-variable warning.

3b/F-51 [P3] closed - A business name with no letters or digits comes back as a form-level notice, not under the Business name field

File: frontend/lib/api-client.ts:177 **Found:** 2026-09-30 by /audit independent (scope: step 3b.3; lens: quality) **Why it matters:** The shared schema only asks for 2 to 80 characters, so a name like "!!" or "--" passes the form, and the API answers `400 bad_request` ("Use at least a couple of letters or numbers in the name."). `provisionClient` maps only `email_taken` and `slug_taken` to fields, so this lands in the form-level `Notice`. The spec's Frontend states keep the form-level error for "API down or a 500" and list field errors separately; the deleted create page checked `toSlug` before sending for exactly this case. Cosmetic: the message still says what to fix. **Suggested fix:** Map `bad_request` to `businessName` in `provisionClient` (it is the only field the server can refuse after the shared schema passed), or check `toSlug(businessName)` in the form before sending, as the create page did. **Resolution:** Fixed 2026-09-30 on Frank's yes, in the step 3b.3 review-fix commit; waits for the next review (the final one at /complete) to close. `businessNameValidationSchema` refuses a name whose address would be empty ("Use at least a couple of letters or numbers."), so the form shows it under Business name and sends nothing; the API's own empty-slug check stays as the backstop. Shared test added; checked in the browser: "!! ??" showed the message under the field and no request was sent. Closed 2026-09-30 by /audit independent (scope: current, final review of feature 3b): `businessNameValidationSchema` (`business-name-validation-schema.ts:7-13`) refines on `toSlug(name) !== ""`, imported from `helpers/`, so the form's `zodResolver` puts the message under Business name before any request; the shared test (`provision-client-validation-schema.test.ts:27`) and the backend route test (`admin-routes.test.ts:142`, `400 bad_request`, no login) cover both ends, and `provisionClient`'s own empty-slug check (`provision-client.ts:30`) stays as the backstop. Shared 56/56, backend 122/122.

Independent review

Status: passed **Target commit:** 42315d9f5736b5bd76ae353233f92627140617d4 **Base commit:** 6624b219a6de5b735baadfe55bf99c0694318c49 **Base ref:** main **Spec hash:** 5df94c97f8522283b24c226960b10748f4c16f0c7b544ee93792f67ce05a6950 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-01T03:40:37Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-01T03:45:00Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`6624b219a6de5b735baadfe55bf99c0694318c49..42315d9f5736b5bd76ae353233f92627140617d4` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain=v1 -uall` : pass (HEAD equals the target, `main` gives the recorded merge base, the spec hash matches, only `review.md` differed)
- `npm run test --workspace=@scheduleads-app/shared` : pass (6 files, 56 tests)
- `npm run test --workspace=backend` : pass (7 files, 122 tests, against local `scheduleads_dev`)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass (`/admin/clients/new` built; `/create-organization` gone)
- `npm run lint --workspace=frontend` : pass
- Read-only query of the local database after the run: no `client_setup_claim` rows, no `admin-%@example.com` logins, no `test-provision-%` businesses left

Evidence

- Whole delta read (36 files, 7 commits `61a7d4b..42315d9`): route, middleware, `provision-client.ts`, claim table and migration 0005, `auth-server.ts`, refusal codes, moved `safe-error-reason.ts`, shared schemas and tests, the admin page, form, no-business card, `sign-out-link.tsx`, `is-platform-admin.ts`, `api-client.ts`, shadcn `input / label`, `auth-card.tsx`
- Security: Better Auth 1.7.5 `crud-org.mjs` `createOrganization` sets `isSystemAction` only when there is no session and a `userId`, so with `allowUserToCreateOrganization: false` any HTTP create (session user) is 403 and the server call works; `admin/routes.mjs` `createUser` skips `hasPermission` only with no session and no headers. `provisionClient` is called only from `admin-routes.ts:249`, behind `requirePlatformAdminMiddleware` and the dashboard CORS, origin check and no-store on `/admin/*` (`app.ts:22`); the body is parsed with the shared schema before the setup's own queries; every name and email renders as React text
- Data exposure: every error out of a setup passes `provisionClient`'s catch and only `safeErrorReason` is logged; Better Auth's drizzle adapter does not rewrap database errors, so a failed query still reduces to its Postgres code

- Performance: no connection is reserved or held across a setup (no `reserve(`, `begin(` or advisory call remains; `database.ts` unchanged from `main`); claims are two-row inserts on a primary key; the 14-at-once test passes and `/health` still answers
- Tests: route tests cover 401, 403, 400, the 201 rows, sign-in and `/me`, both conflicts, reuse, three forced failures, overlap with claims, a foreign claim, a stale claim, the log redaction, the guards and the two closed Better Auth doors; each cleans its own tagged rows
- Ledger: F-13, F-38, F-42, F-48, F-49, F-50, F-51 re-examined and closed with evidence; F-14, F-32, F-34, F-47 re-read and left as they are

Findings

- F-52 [P3] open (new): three comments and one standard still point at the advisory lock and at a role check that moved (`provision-client.ts:171`, `require-platform-admin-middleware.ts:15`, `coding-standards.md:402-403`)
- Closed this pass: F-13, F-38, F-42, F-48, F-49, F-50, F-51
- Unchanged: F-14 [P3] open, F-32 [P3] unverified, F-34 [P3] open, F-47 [P3] open
- No P0 or P1 is open or fixed

Remaining risk

- No `Verify` command and no GitHub CI check exist (`AGENTS.md`, Commands); the signals above were run by hand
- The frontend has no test runner, and no `Browser tests` command exists, so the admin page, the form's field errors, the success card, the no-business card and the Sign out footers were reviewed by reading plus the build and lint, not run in a browser by this reviewer
- `/check` was not required and was not run
- A setup still running after five minutes can have its claims cleared by the next setup (the spec's stated limit); a setup has no reason to run that long, but no timeout bounds it
- If a setup's clean-up itself fails, the half-made setup stays and a retry is `email_taken` until feature 23 can delete a client (the spec's stated limit)
- F-47: a pending invitation made before `invitation: []` stays acceptable until it expires; the production `invitation` table must be checked before the first client-facing deploy
- Production sign-in cannot send a login code until feature 6 (Resend); unchanged by this feature